

3507. Minimum Pair Removal to Sort Array I

Solved 

Easy

 Topics

 Companies

 Hint

Given an array `nums`, you can perform the following operation any number of times:

- Select the **adjacent** pair with the **minimum** sum in `nums`. If multiple such pairs exist, choose the leftmost one.
- Replace the pair with their sum.

Return the **minimum number of operations** needed to make the array **non-decreasing**.

An array is said to be **non-decreasing** if each element is greater than or equal to its previous element (if it exists).

Example 1:

Input: `nums = [5,2,3,1]`

Output: 2

Explanation:

- The pair `(3, 1)` has the minimum sum of 4. After replacement, `nums = [5,2,4]`.
- The pair `(2, 4)` has the minimum sum of 6. After replacement, `nums = [5,6]`.

The array `nums` became non-decreasing in two operations.

Example 2:

Input: `nums = [1,2,2]`

Output: 0

Explanation:

The array `nums` is already sorted.

Constraints:

- `1 <= nums.length <= 50`
- `-1000 <= nums[i] <= 1000`

Python:

class Solution:

```
def minimumPairRemoval(self, nums: List[int]) -> int:
    def isSorted(nums, n) -> bool:
        for i in range(1,n):
            if nums[i] < nums[i - 1]: return False
        return True
    ans, n = 0, len(nums)
    while not isSorted(nums, n):
        ans += 1
        min_sum, pos = float('inf'), -1
        for i in range(1,n):
            sum = nums[i - 1] + nums[i]
            if sum < min_sum:
                min_sum = sum
                pos = i
        nums[pos - 1] = min_sum
        for i in range(pos, n-1): nums[i] = nums[i + 1]
        n -= 1
    return ans
```

JavaScript:

```
var isSorted = function(nums, n) {
    for(let i = 1; i < n; i++) {
        if(nums[i] < nums[i - 1]) return false;
    }
    return true;
}
var minimumPairRemoval = function(nums) {
    let ans = 0, n = nums.length;
    while(!isSorted(nums, n)) {
        ans += 1;
        let min_sum = Infinity, pos = -1;
        for(let i = 1; i < n; i++) {
            let sum = nums[i - 1] + nums[i];
            if(sum < min_sum) {
                min_sum = sum;
            }
        }
        nums[pos] = min_sum;
        for(let i = pos + 1; i < n; i++) {
            nums[i - 1] = nums[i];
        }
        n--;
    }
    return ans;
}
```

```

        pos = i;
    }
}
nums[pos - 1] = min_sum;
for(let i = pos; i < n - 1; i++) nums[i] = nums[i + 1];
n--;
}
return ans;
};

```

Java:

```

class Solution {
    private boolean isSorted(int[] nums, int n) {
        for(int i = 1; i < n; i++) {
            if(nums[i] < nums[i - 1]) return false;
        }
        return true;
    }
    public int minimumPairRemoval(int[] nums) {
        int ans = 0, n = nums.length;
        while(!isSorted(nums, n)) {
            ans += 1;
            int min_sum = Integer.MAX_VALUE, pos = -1;
            for(int i = 1; i < n; i++) {
                int sum = nums[i - 1] + nums[i];
                if(sum < min_sum) {
                    min_sum = sum;
                    pos = i;
                }
            }
            nums[pos - 1] = min_sum;
            for(int i = pos; i < n - 1; i++) nums[i] = nums[i + 1];
            n--;
        }
        return ans;
    }
}

```